IMPLEMENTATION OF BLOCKS WORLD PROGRAM

Aim:

To implement binary search tree.

PROGRAM:

```
class BlocksWorld:
    def __init__(self, num_blocks):
        self.state = [[block] for block in range(num_blocks)]

    def display_state(self):
        for stack in self.state:
            if stack:
                print("Block(s) on stack:", stack)

    def find_block(self, block):
        for stack in self.state:
            if block in stack:
                return stack
        return None

    def move(self, block, destination):
        source_stack = self.find_block(block)

        if source_stack is None:
            print("Invalid block:", block)
```

```
            return

        if source_stack[-1] != block:
            print("Cannot move block", block, "- another block is on
top.")
            return

        if destination == "table":
            source_stack.remove(block)
            self.state.append([block])
        else:
            destination_stack = self.find_block(destination)

            if destination_stack is None:
                print("Invalid destination:", destination)
                return

            if destination_stack[-1] != destination:
                print("Cannot place block on", destination)
                return

            if block == destination:
                print("A block cannot be placed on itself.")
                return

            source_stack.remove(block)
            destination_stack.append(block)

        self.state = [stack for stack in self.state if stack]
        print("\nAfter moving", block, "to", destination, ":")
```

```
        self.display_state()


def main():
    blocks_world = BlocksWorld(3)

    print("Initial state:")
    blocks_world.display_state()

    print("\nPerforming moves:")
    blocks_world.move(0, 2)
    blocks_world.move(0, "table")
    blocks_world.move(1, 2)


if __name__ == "__main__":
    main()
```

OUTPUT:

```
Initial state:
Block(s) on stack: [0]
Block(s) on stack: [1]
Block(s) on stack: [2]

Performing moves:

After moving 0 to 2 :
Block(s) on stack: [1]
Block(s) on stack: [2, 0]
```

After moving 0 to table :
Block(s) on stack: [1]
Block(s) on stack: [2]
Block(s) on stack: [0]

After moving 1 to 2 :
Block(s) on stack: [2, 1]
Block(s) on stack: [0]